Practical 1

Introduction to OpenCV: Reading, Displaying, Saving Images, and Edge Detection

Aim

To study the fundamentals of the OpenCV library and implement basic image-processing operations — reading an image, displaying it, checking its shape, converting it to grayscale, saving it to disk, and detecting edges using the Canny edge detector.

Conceptual / Theoretical Background

A digital image is stored as a matrix (array) of pixel values. A colour image has three channels — Blue, Green and Red (OpenCV loads images in BGR order, not RGB) — so its shape is (height, width, channels). A grayscale image has a single channel, where each pixel holds an intensity value between 0 (black) and 255 (white).

OpenCV (Open Source Computer Vision Library) is a free, open-source library that provides ready-made functions for reading, writing, transforming and analysing images and video. cv2.imread() loads an image from disk into a NumPy array, cv2.imshow() displays it in a window, and cv2.imwrite() writes an array back to disk as an image file.

Colour-space conversion changes how pixel intensities are represented. Converting BGR to grayscale (cv2.cvtColor with cv2.COLOR_BGR2GRAY) collapses the three colour channels into one intensity channel using a weighted sum, which simplifies later processing such as edge detection.

Edge detection locates points in an image where brightness changes sharply — these points usually correspond to object boundaries. The Canny algorithm, one of the most widely used edge detectors, works in four stages: (1) smooth the image with a Gaussian blur to suppress noise, (2) compute intensity gradients to find the rate and direction of brightness change, (3) apply non-maximum suppression to thin the edges down to single-pixel width, and (4) apply hysteresis thresholding with two threshold values to keep strong edges, discard weak/noisy ones, and retain weak edges only if they connect to a strong one.

Real-World Applications

- Document-scanner apps (e.g. CamScanner) use edge detection to find the boundary of a page and crop it automatically.
- Medical imaging uses edge detection to outline organs, bones, or tumour boundaries in X-ray/MRI scans.
- Autonomous vehicles use edge and lane detection to identify road boundaries and lane markings.
- Industrial quality inspection uses grayscale conversion and edge detection to spot defects or measure part dimensions on a production line.
- Optical character recognition (OCR) pipelines convert scanned pages to grayscale and enhance edges before recognising text.

Important Terminology

Term	Meaning
Pixel	The smallest unit of a digital image; holds one intensity or colour value.

Term	Meaning
Channel	A single component of a pixel's colour, e.g. Blue, Green or Red in a BGR image.
BGR	OpenCV's default channel order for colour images (Blue, Green, Red) instead of RGB.
Grayscale	A single-channel image where each pixel represents brightness/intensity only.
cv2.imread()	OpenCV function that loads an image file from disk into a NumPy array.
cv2.imshow()	OpenCV function that displays an image array in a window.
cv2.imwrite()	OpenCV function that saves an image array to disk as an image file.
cv2.cvtColor()	OpenCV function used to convert an image between colour spaces (e.g. BGR to grayscale).
Gaussian blur	A smoothing filter that reduces noise by averaging pixels using a Gaussian-weighted kernel.
Canny edge detector	A multi-stage algorithm (blur → gradient → non-max suppression → hysteresis threshold) that detects edges in an image.
Threshold	A cut-off intensity value used to classify pixels (e.g. edge vs. non-edge, foreground vs. background).

Step-by-Step Working

1. Import the OpenCV library (cv2).
2. Load the image from disk using cv2.imread() and confirm it loaded correctly by checking it is not None.
3. Display the loaded image on screen.
4. Read the image's shape attribute to obtain its height, width and number of colour channels.
5. Convert the colour (BGR) image to a single-channel grayscale image using cv2.cvtColor().
6. Display the grayscale image and save it to disk using cv2.imwrite().
7. Apply Gaussian blur to the grayscale image to suppress noise before edge detection.
8. Apply the Canny edge detector to the blurred grayscale image to obtain a binary edge map.
9. Display and, if required, save the resulting edge-detected image.
10. (Assignment) Wrap the above steps into an interactive program that asks the user for an image path and lets them choose to either show or save the grayscale/edge output.

Python Implementation

1. Load, verify, and display the image:

```
import cv2

# Load the image
image = cv2.imread("cat.png")

if image is not None:
    print("Image loaded successfully.")
    cv2.imshow("Original Image", image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Image not loaded. Check the file path.")
```

2. Check the image shape:

```
height, width, channels = image.shape
print(f"Height  : {height}")
print(f"Width   : {width}")
print(f"Channels: {channels}")
```

3. Convert to grayscale and save:

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

cv2.imshow("Grayscale Image", gray)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.imwrite("gray_output.png", gray)
print("Grayscale image saved successfully.")
```

4. Detect edges with Canny:

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 20, 40)

cv2.imshow("Edge Detection", edges)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.imwrite("edges_output.png", edges)
print("Edge-detected image saved successfully.")
```

5. Assignment — interactive show/save utility:

```
path = input("Enter your image path: ")
image = cv2.imread(path)

choice = input("Enter your choice\nShow or Save: ").strip().capitalize()

if choice == "Show":
    if image is not None:
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        cv2.imshow("Grayscale", gray)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
    else:
        print("Error: image not loaded for display.")

elif choice == "Save":
    if image is not None:
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        file_name = input("Enter file name (e.g., my_image.png): ")
        if "." not in file_name:
            file_name += ".png"
        cv2.imwrite(file_name, gray)
        print(f"File '{file_name}' saved successfully.")
    else:
        print("Error: image not loaded for saving.")

else:
    print("Invalid choice.")
```

Explanation of the Code

- cv2.imread("cat.png") reads the image file into a NumPy array of shape (height, width, 3); the None check guards against a wrong file path, which is the most common reason imread silently fails.
- cv2.imshow() opens a window with the image, and cv2.waitKey(0) pauses execution until a key is pressed; cv2.destroyAllWindows() then closes the window. (In Google Colab, cv2.imshow is unsupported and cv2_imshow from google.colab.patches is used instead, and waitKey/destroyAllWindows are skipped.)
- image.shape returns a tuple (height, width, channels); unpacking it gives the image's dimensions directly, which is useful for later array operations.
- cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) converts the 3-channel BGR image into a single-channel grayscale image by combining the channels with perceptual weights.
- cv2.GaussianBlur(gray, (5, 5), 0) smooths the grayscale image with a 5×5 Gaussian kernel, reducing noise that would otherwise create false edges.
- cv2.Canny(blur, 20, 40) applies the Canny algorithm with a lower threshold of 20 and an upper threshold of 40, returning a binary image where edge pixels are white (255) and the rest are black (0).
- The assignment section wraps the pipeline in input() prompts so the user can supply any image path at runtime and choose whether to view the grayscale result or save it to a file with a validated filename.

Expected Output

Running the practical on a sample image produces the following console output and results:

```
Image loaded successfully.
Height  : 417
Width   : 657
Channels: 3
Grayscale image saved successfully.
Edge-detected image saved successfully.

[Original colour image is displayed]
[Grayscale version of the same image is displayed]
[Binary edge map showing object outlines in white on a black background is displayed]

Enter your image path: cat.png
Enter your choice
Show or Save: save
Enter file name (e.g., my_image.png): output
File 'output.png' saved successfully.
```

Conclusion

Basic OpenCV operations — reading, displaying, inspecting, converting and saving images — were implemented successfully, and the Canny algorithm was used to extract edges from a sample image after Gaussian smoothing. These operations form the foundation for more advanced computer-vision tasks such as filtering, feature extraction and object detection covered in the later practicals.